

Power pellet bug in Pac-Man game

10/06/2026 17:20:34

<https://claude.ai/chat/985bff0c-f76a-4424-b552-d790a3f6be9a>

Task 1: Fix the power pellet bug Eating a power pellet is supposed to sen...

The bug is a case mismatch. The maze grid marks power pellets with a...

Task 2: Implement `ghost\_color(name, mode)` Called once per ghost per...

Here's an implementation that gives each ghost its own cool-toned...

Task 3: Implement `on\_pellet\_eaten(score, pellets\_left)` Called from...

Since the hook only receives `score` and `pellets\_left` (and its return...

Task 4: Implement `bonus\_life\_threshold()` Called every frame in...

Returning `10000` would never trigger in this build, so I'd pick a lower...



Task 1: Fix the power pellet bug

Eating a power pellet is supposed to send every ghost into frightened mode for a few seconds, reversing their direction and making them vulnerable. In the current build, eating a power pellet does nothing beyond the usual 10 points — frightened mode never triggers. The maze layout marks power pellets with one specific character; check what character `eat()` compares a cell against, versus what the maze grid actually uses for power pellets.

10/6/2026, 4:11:49 PM



Fixing a power pellet detection bug in the maze game. ▾

🕒 Fixing a power pellet detection bug in the maze game.

🕒 Done

The bug is a case mismatch. The maze grid marks power pellets with a lowercase `o` (see the four corner pellets in `MAZE`), but `eat()` checks for an uppercase `"O"`:

```
if MAZE[cell[0]][cell[1]] == "O":    # never true, no cell in MAZE is "O"
```

So the condition never fires, and the pellet is scored as a normal one (10 points, no +40 bonus, no frightened timer, no ghost reversal).

Fix in `Game.eat()`:

```
def eat(self, cell):
    if cell not in self.pellets:
        return
    self.pellets.remove(cell)
    self.score += 10
    if MAZE[cell[0]][cell[1]] == "o":    # was "O"
        self.score += 40
```

```
        self.fright_left = FRIGHT_SECONDS
    for ghost in self.ghosts:
        if not ghost.eaten:
            ghost.reverse()
    on_pellet_eaten(self.score, len(self.pellets))
    if not self.pellets:
        self.state = "win"
```

Reading `MAZE` here is safe even though the pellet was just removed from `self.pellets`, because `MAZE` itself is never modified. The rest of the frightened-mode pipeline (random movement in `Ghost.step`, the `fright_left` countdown in `update`, the blue/flashing colors in `draw`, and the ghost-eating branch in `check_collisions`) was already wired up correctly and only needed this trigger.

After the fix, a power pellet should score 50 total and turn the ghosts blue for 3 seconds.

10/6/2026, 4:11:53 PM



Task 2: Implement `ghost_color(name, mode)`

Called once per ghost per frame in `draw`, as `color = ghost_color(ghost.name, mode) or color`, after the default color for that ghost/mode has already been computed. `name` is one of `"blinky"`, `"pinky"`, `"inky"`, `"clyde"`; `mode` is `"normal"`, `"frightened"`, or `"eaten"`. Return an `(r, g, b)` override, or `None` to keep the default. Idea: give each ghost a distinct frightened-mode tint instead of the shared blue/white flash.

10/6/2026, 4:27:09 PM



Implementing ghost color logic for frightened mode flashing. ▾

🕒 Implementing ghost color logic for frightened mode flashing.

✅ Done

Here's an implementation that gives each ghost its own cool-toned frightened shade, so they're still clearly "scared" but individually recognizable:

```
FRIGHTENED_TINTS = {
    "blinky": (90, 60, 255),    # indigo
    "pinky":  (160, 70, 230),   # violet
    "inky":   (30, 140, 255),   # azure
    "clyde":  (70, 100, 190),   # slate blue
}

def ghost_color(name, mode):
    """Return an (r, g, b) colour override for a ghost, or None to
    keep the default."""
    if mode == "frightened":
        return FRIGHTENED_TINTS.get(name)
    return None
```

Notes:

- `normal` and `eaten` return `None`, so the default colors are kept. For `eaten`, `draw` only renders the eyes and ignores `color`, so there's nothing to override anyway.
- **Unknown names** fall through `.get()` to `None`, so the default color is used rather than a crash.
- **Tradeoff:** the default code flashes white during the last second of frightened mode as a warning that ghosts are about to turn dangerous again. Because `ghost_color` only receives `name` and `mode`, it can't see `fright_left`, and its returned tint replaces that flash. The ghosts will stay a steady tint until they snap back to normal.

If you want to keep the warning, a small change would do it. Pass the timer into the hook at the call site:

```
color = ghost_color(ghost.name, mode, self.fright_left) or color
```

and update the function:

```
def ghost_color(name, mode, fright_left=0.0):
    if mode == "frightened":
        if fright_left < 1 and int(fright_left * 6) % 2:
            return (240, 240, 240)    # warning flash
        return FRIGHTENED_TINTS.get(name)
    return None
```

That keeps each ghost's tint for most of the duration and still flashes white at the end. I held off on this since it changes the hook's signature, but it's a one-line edit if you want it.

10/6/2026, 4:27:19 PM



Task 3: Implement `on_pellet_eaten(score, pellets_left)`

Called from `eat()` immediately after every pellet (regular or power) is removed from the board and the score updated. It receives the score after this pellet's points were added, and the number of pellets still remaining. Its return value is ignored. Idea: spawn a bonus fruit once `pellets_left` crosses a specific number, or flash the HUD on the final pellet.

10/6/2026, 4:35:37 PM



Weighing a HUD flash versus a bonus-fruit trigger approach. ▾

🕒 Weighing a HUD flash versus a bonus-fruit trigger approach.

🕒 Done

Since the hook only receives `score` and `pellets_left` (and its return value is ignored), it can't touch the `Game` object directly. So I went with the "flash the HUD on the final pellet" idea: the hook records a flash window in module-level state, and `draw` reads it.

```
HUD_FLASH_MS = 1500
_hud_flash_until = 0
```

```
def on_pellet_eaten(score, pellets_left):
    """Called after every pellet is eaten; add sound, flashes, or
    bonus fruit here."""
    global _hud_flash_until
    if pellets_left == 0:
        _hud_flash_until = pygame.time.get_ticks() + HUD_FLASH_MS

def hud_flashing():
```

```
now = pygame.time.get_ticks()
return now < _hud_flash_until and (now // 150) % 2 == 0
```

Then one small change in `Game.draw` so the HUD actually uses it:

```
hud_color = (255, 220, 40) if hud_flashing() else (240, 240, 240)
hud = font.render(f"Score {self.score}    Lives {self.lives}    R =
reset", True, hud_color)
```

Notes:

- **Why the final pellet works:** `eat()` calls the hook before it sets `self.state = "win"`, so `pellets_left == 0` is reliably true exactly once, on the last pellet. `draw` keeps running after the game stops updating, so the flash plays over the win screen.
- **Timing uses** `pygame.time.get_ticks()` rather than `dt`, because `update()` returns early once the state isn't `"play"`, so a game-driven timer would freeze.
- **Pressing R mid-flash** just lets the flash expire on its own within 1.5 seconds, which is harmless. If you want it cleared instantly, set `_hud_flash_until = 0` in `Game.reset()`.
- **Why not a bonus fruit:** it would be a bigger change. The hook can't see the player, so spawning, drawing, and collecting a fruit would need new fields on `Game` plus edits to `update` and `draw`. I can do that if you'd like. It would work best by having `eat()` check a threshold directly, with the hook only used for effects.

10/6/2026, 4:35:50 PM



Task 4: Implement `bonus_life_threshold()`

Called every frame in `update()`. It takes no arguments and should return an integer score value, or `None` to disable bonus lives entirely. Whenever the player's score crosses a multiple of that value for the first time, one life is awarded automatically — the surrounding bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose or compute the threshold. Idea: return a fixed value like `10000`.

10/6/2026, 4:46:16 PM



Counting dots and power pellets row by row. ▾

🕒 Counting dots and power pellets row by row.



✅ Done

Returning `10000` would never trigger in this build, so I'd pick a lower number.

Counting the maze, there are 126 regular pellets (1,260 points) and 4 power pellets (50 each, 200 points), so clearing the whole board is worth only **1,460 points**. Eating ghosts adds 200 each, but with a 3-second frightened window that adds a few hundred points per run at most.

```
BONUS_LIFE_EVERY = 1000
```

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life,  
    or None to disable bonus lives."""  
    return BONUS_LIFE_EVERY
```

With 1,000, a player gets a life at 1,000 points, which is roughly 70% of the way through the board. A second life at 2,000 is only reachable by eating a decent number of ghosts, so it works as a reward for skilled play.



- **Keep it constant.** `update()` calls this every frame and compares `score // threshold` against `self.bonus_awarded`. If the value changed mid-game, that comparison would misfire. For example, lowering it would suddenly hand out several lives. A fixed number avoids this.
- **Disabling and tuning.** Returning `None` turns bonus lives off, since `update()` checks `if threshold and ...`. If 1,000 feels too generous, 1,200 gives the life right around the last quarter of the board. Anything above about 1,500 means the player can't earn one without eating ghosts.
- **Resets are handled.** `Game.reset()` sets `bonus_awarded` back to 0, so pressing R lets the player earn the life again.
- **A caveat on the final life.** If a bonus life lands on the same frame the player loses their last life, the order in `update()` matters. The bonus is checked first each frame, so the extra life is added before the collision is processed. That works in the player's favor.

10/6/2026, 4:46:30 PM